Escape the Maze — Work Plan, Task Split & Timeline

AI Mini Project 01 · 30 marks (Code 25 + Report 5) · Due **14 October 2026, 11:59 pm** · Plan written 1 October 2026

1. Snapshot

Item	Detail
Team	Nofil Ali Khan (repo owner, merges PRs) + Partner (works on a fork)
Repository	`github.com/Nofil2003/maze-runner` — `main` is the shared trunk
Local path	`~/Documents/github/maze_runner/maze_runner` (project root, where `runner.py` lives)
Files we edit	`search.py` (Q1–Q7), `heuristics.py` (Q6), `griever_hole.py` (Q8–Q9)
Files we only read	`problems.py`, `util.py`, `maze.py`
Submission	`<group_number>_project1.zip` on GCR containing the 3 `.py` files + `report.pdf`. One member submits.
Assessment	Code 25 marks, Report 5 marks, **plus a viva** on our understanding
Effort estimate	≈ 28–32 person-hours total, ≈ 15–16 hours each
Internal deadline	**Submit Monday 13 October.** 14 October is buffer only.

Marks per question

Q	Topic	Marks	Owner	Complexity
Q1	Depth-First Search	2	Nofil	Low (but carries the shared helper)
Q2	Breadth-First Search	2	Nofil	Low
Q3	Depth-Limited Search	3	Partner	**Medium–High**
Q4	Iterative Deepening Search	2	Partner	Low
Q5	Uniform-Cost Search	3	Nofil	Low–Medium
Q6	Heuristics + Greedy Best-First	3	Nofil	Low
Q7	A* Search	3	Nofil	Low

Q	Topic	Marks	Owner	Complexity
Q8	KeyHuntProblem (formulation)	3	Partner	**Medium**
Q9	Key-hunt heuristic	4	**Both, paired**	**High**
R	Report	5	Split (see §7)	Medium

Code marks by owner: Nofil ≈ 13, Partner ≈ 8, Q9's 4 marks shared. The split is **by file and by function** so that two people never edit the same lines.

> **Two hard rules from the brief.** (1) The report must state which AI assistant we used and for what — so keep an honest running note of it from day one. (2) **Any screenshot of code in the report = 0 marks for the report.** Describe things in prose and tables instead.

2. Ground rules

Conventions every algorithm must follow

These come straight from "Rules of the Maze" and the grader checks them. Getting one wrong silently breaks the expected numbers:

- An algorithm returns a **list of actions**; `[]` if the start is already a goal; `None` if there is no solution. DLS may also return `CUTOFF`.
- **Goal-test on pop**, not when a successor is generated.
- Push successors **in the order `getSuccessors` returns them** (North, South, East, West).
- Call `getSuccessors(state)` **exactly once per expanded state** — it is what counts an expansion.
- Mark a state expanded **when you pop and expand it**, not when you push it.
- Q1, Q2, Q5, Q6, Q7 are **graph searches** with an explored set. **Q3 must NOT use a global explored set** — it avoids cycles by skipping states already on the current path.

Git workflow

- `main` is treated as protected: **no direct pushes**, PRs only, Nofil merges.
- One branch per question: `q1-dfs`, `q3-dls`, `q8-keyhunt`, `q9-heuristic`, …
- Partner syncs their fork from upstream `main` **before** starting each new branch.
- Nobody reformats or restructures a file they do not own. No force-pushes to shared branches.
- Every PR runs `verify.sh` for the rows it affects, and the output goes in the PR description.
- Commit messages: `qN: <what changed>`, e.g. `q3: recursive DLS with path-based cycle check`.
- Repo hygiene first: add a `.gitignore` and remove `__MACOSX/`, `.DS_Store`, `__pycache__/`, `.Rhistory` from tracking.

Running things

The repo root holds `verify.sh`, which runs every row of the table below and checks the numbers:

```
./verify.sh             # every row
./verify.sh q3          # only question 3's rows
./verify.sh q1 q2 q5    # several questions
```

It prints PASS / WARN / FAIL per row, plus `TODO` for anything still unimplemented, and exits non-zero if anything fails. WARN means the node count differs slightly, which the brief allows; a difference of more than 25% is reported as FAIL. It is a development tool and is **not** part of the submission zip.

On machines where `python` is not on the PATH (Ubuntu, for example), use `python3 runner.py …`; `verify.sh` already does, and honours a `PYTHON=` override.

Reference table we must reproduce exactly

`verify.sh` automates this. Path cost and steps must match **exactly** for BFS, IDS, UCS and A* (they are optimal). For DFS and greedy, any legal path is correct, but with our conventions we should see these numbers. Node counts may differ slightly; a large difference means a bug.

Q	Command (`python runner.py …`)	Cost	Steps	Expanded
1	`-l maze_small -a dfs`	50	50	82
1	`-l maze_medium -a dfs`	162	162	233
2	`-l maze_tiny -a bfs`	8	8	27
2	`-l maze_medium -a bfs`	66	66	407
2	`-l open_glade -a bfs`	32	32	344
3	`-l maze_small -a dls -d 60`	50	50	52
3	`-l maze_small -a dls -d 30`	CUTOFF	—	32
4	`-l maze_small -a ids`	50	50	1352
5	`-l griever_alley -a ucs`	42	28	50
5	`-l ivy_maze -a ucs`	70	64	418
6	`-l greedy_trap -a gbfs`	52	25	60
6	`-l maze_medium -a gbfs`	80	80	81
7	`-l greedy_trap -a astar`	25	25	155
7	`-l maze_big -a astar`	124	124	679
8	`-l keys_tiny -p keys -a bfs`	26	26	490
8	`-l keys_small -p keys -a bfs`	54	54	1723

3. Responsibility split

Nofil (owner, merges)	Partner (fork → PR)
Repo hygiene, `.gitignore`, `verify.sh` harness	Day-1 read of `problems.py`, `util.py`, `maze.py`, `alwaysRightSearch`
Q1 DFS + the shared graph-search helper — pushed to `main` on day 2 so the partner can branch off it	**Q3 Depth-Limited Search** (recursive, independent of the helper)
Q2 BFS	**Q4 Iterative Deepening Search**
Q5 UCS	**Q8 KeyHuntProblem** — lives in its own file, so zero merge conflicts
Q6 Manhattan + Euclidean heuristics, Greedy Best-First	`--compare` runs on `maze_medium`, `open_glade`, `ivy_maze`; collect tables
Q7 A*	Report drafts: §3 DFS vs BFS, §4 IDS, §5 Costs

Nofil (owner, merges)	Partner (fork → PR)
Report: §1 AI disclosure, §2 results tables, §6 heuristics, §7 Q9 write-up; final typesetting to PDF	Proof-read the final report; verify the zip contents
Q9 heuristic — paired session, both of us	**Q9 heuristic — paired session, both of us**

Why this split: `search.py` is one file, so we divide it by *function*, not by line. Nofil owns the five algorithms that share one graph-search loop (Q1, Q2, Q5, Q6, Q7) — writing them together as one helper is faster than splitting them and avoids five people-sized conflicts. The partner owns the two recursive algorithms (Q3, Q4) and the whole of `griever_hole.py` (Q8), which touch separate regions and separate files. Q9 is the hardest item and the most likely viva target, so we do it together rather than hand it to one person.

4. Timeline

Date	Nofil	Partner	Milestone
Wed 1 Oct	Repo cleanup, `.gitignore`, write `verify.sh`, push conventions to `main`	Clone fork, read the 3 read-only files, run `runner.py -l maze_tiny -a right`	Repo clean, both can run the project
Thu 2 Oct	Q1 DFS + shared helper → PR → **merge to `main`**	Start Q3 DLS	Helper on `main`
Fri 3 Oct	Q2 BFS, Q5 UCS	Finish Q3 (both `-d 60` and `-d 30` rows pass)	Q1–Q3 green
Sat 4 Oct	Q6 heuristics + GBFS	Q4 IDS	Q4 green
Sun 5 Oct	Q7 A*	Q8 state design, then implement	Q1–Q7 green
Mon 6 Oct	Merge everything, run the **full** verify table on `main`	Q8 hits 26 / 54 steps on `keys_tiny` / `keys_small`	**All of Q1–Q8 on `main` and verified**
Tue 7 Oct	Q9 paired: baseline consistent heuristic	Q9 paired	h > 0 at start, consistent → 1 mark secured
Wed 8 Oct	Q9 paired: push under 2600 expansions	Q9 paired	3 marks secured
Thu 9 Oct	Q9 paired: push under 1200 expansions	Q9 paired	**4 marks secured**
Fri 10 Oct	Report §1, §2, §6, §7	Report §3, §4, §5 drafts; run the three `--compare` tables	Full report draft exists
Sat 11 Oct	Assemble + typeset report → `report.pdf`	Proof-read, check no code screenshots	`report.pdf` final
Sun 12 Oct	**Viva prep:** each of us explains the *other's* code aloud	Viva prep, same	Both can defend all 9 answers
Mon 13 Oct	Fresh clone, `verify.sh` green, build zip, **submit on GCR**	Review zip contents before submit	**Submitted**

Date	Nofil	Partner	Milestone
Tue 14 Oct	Buffer — do not plan work here	Buffer	Deadline 11:59 pm

Slack built in: one full buffer day (14 Oct), plus 6 Oct is an integration day that can absorb a slipped question.

5. What each question involves, and how to prepare for it

Q0 — Setup (both, day 1, ~1.5 h)

What gets done. Read `problems.py` (the `SearchProblem` interface, `MazeProblem`, `GraphProblem`, `mazeDistance`), `util.py` (`Stack`, `Queue`, `PriorityQueue`, `CUTOFF`, `manhattanDistance`), `maze.py` (terrain costs, neighbour order), and `alwaysRightSearch` in `search.py` — it is a complete worked example of how an algorithm talks to a problem. Run `python runner.py -l maze_tiny -a right` (escapes) and `-l maze_small -a right` (walks into a wall). Nofil additionally writes `verify.sh`: a script that runs every row of §2's table with `-q` and diffs against the expected numbers. It is a development tool and is **not** part of the submission.

How to prepare. Memorise the four-method interface — `getStartState()`, `isGoalState(state)`, `getSuccessors(state)`, `getCostOfActions(actions)` — because nothing we write may touch the maze directly. Learn the terrain costs cold: wall `%` impassable, corridor/`S`/`E`/`K` cost 1, ivy `~` cost 3, Griever territory `G` cost 10, and the cost is for the cell you **step into**. Note that positions are `(row, col)` with row 0 at the top, and actions are `'North'`, `'South'`, `'East'`, `'West'`. If the graphics window fails, use `-t` for text output, or `sudo apt install python3-tk`.

Q1 — Depth-First Search · 2 marks · Nofil · ~2 h

What gets done. Implement `depthFirstSearch` as a **graph** search with a `Stack` fringe. Each fringe entry stores `(state, actionsSoFar)` so the path comes back for free. The real work here is factoring out a shared `graphSearch(problem, fringe, ...)` loop that Q2, Q5, Q6 and Q7 will all reuse — pop, goal-test, skip if already expanded, mark expanded, push children.

How to prepare. Be able to write the generic graph-search loop from memory (it is in the brief's "one algorithm, many fringes" box). Understand why DFS is complete on a finite graph with an explored set but is **not** optimal. Be ready to answer: what does DFS do differently from tree search here, and why does the explored set matter?

Done when. `maze_small` → cost 50, 50 steps, 82 expanded; `maze_medium` → 162, 162, 233.

Q2 — Breadth-First Search · 2 marks · Nofil · ~30 min

What gets done. Same loop, `Queue` fringe instead of `Stack`. It must return a path with the **fewest steps**. Critically, it must not assume states are positions: in Q8 the identical function runs on states that are `(position, keys-remaining)` tuples.

How to prepare. Know why BFS is optimal **only when every step costs the same** — which is exactly why it loses to UCS on `griever_alley`. Check your implementation never calls anything maze-specific.

Done when. `maze_tiny` → 8, 8, 27; `maze_medium` → 66, 66, 407; `open_glade` → 32, 32, 344.

Q3 — Depth-Limited Search · 3 marks · Partner · ~3 h

What gets done. `depthLimitedSearch(problem, limit)` as a **recursive** DFS that never goes deeper than `limit` actions, with a three-way return: a list of at most `limit` actions on success, `CUTOFF` when the depth limit stopped the search somewhere (so a deeper solution may exist), or `None` when no goal

was found and the limit was never hit — which proves there is no solution at all. The start state is depth 0. A state at depth `limit` **may be goal-tested but must not be expanded**.

How to prepare. This is the trickiest of the easy questions; two traps to internalise before writing code. **(1) No global explored set.** The brief's counter-example: graph `A→B, A→C, B→C, C→D, D→G` with limit 3. DFS reaches C via B at depth 2, so D sits at depth 3 and gets cut off; if C is then marked "explored" globally, the route `A→C→D→G` that fits the limit exactly is never tried. So avoid cycles by skipping any successor **already on the current path**. **(2) Propagating CUTOFF correctly.** A node returns `CUTOFF` if *any* child returned `CUTOFF` and none found a goal; it returns `None` only if *every* child returned `None`. Compare with `result is CUTOFF`, never `==`. The runner raises Python's recursion limit to 20,000; if you'd rather avoid recursion, use an explicit `Stack` whose entries carry the current path.

Done when. `maze_small -d 60` → cost 50, 50 steps, 52 expanded; `maze_small -d 30` → `CUTOFF`, 32 expanded; `maze_tiny -d 8` runs.

Q4 — Iterative Deepening Search · 2 marks · Partner · ~45 min

What gets done. Call `depthLimitedSearch` with limits 0, 1, 2, … and stop as soon as the result is **not** `CUTOFF` — that is either a solution, or `None`, which proves no solution exists at any depth.

How to prepare. Understand the trade: IDS returns a fewest-steps path like BFS but uses only `O(depth)` memory, paying for it by re-expanding the shallow layers every iteration. Then run it on `open_glade` and watch it die against the 60-second `--timeout` — be ready to explain why an **open** maze is catastrophic for IDS (the branching factor is high and the path-only cycle check cannot prune the many re-routings to the same cell) while a **corridor** maze like `maze_small` is fine. That explanation is report §4.

Done when. `maze_small -a ids` → 50, 50, 1352.

Q5 — Uniform-Cost Search · 3 marks · Nofil · ~1.5 h

What gets done. `uniformCostSearch` on the shared loop with a `PriorityQueue` keyed on the path cost `g(n)`, returning the **cheapest** path, not the shortest.

How to prepare. Know how `util.PriorityQueue` behaves: it pops the lowest priority and breaks ties first-in-first-out, and it does **not** re-prioritise an item already inside it. Two legal strategies — push a state again with its better cost and skip it when popped a second time, or use `update()` for decrease-key. Be ready to explain the `griever_alley` disagreement: BFS takes 20 steps straight through Griever territory for cost 56; UCS takes 28 steps around it for cost 42.

Done when. `griever_alley` → cost 42, 28 steps, 50 expanded; `ivy_maze` → 70, 64, 418.

Q6 — Heuristics + Greedy Best-First · 3 marks · Nofil · ~1.5 h

What gets done. In `heuristics.py`: `manhattanHeuristic` = `|Δrow| + |Δcol|` and `euclideanHeuristic` = the straight-line distance, both taking `(state, problem)` with the goal at `problem.goal`. In `search.py`: `greedyBestFirstSearch`, the shared loop with a `PriorityQueue` keyed on `h(n)` alone.

How to prepare. Have the definitions of **admissible** (never overestimates the true cost) and **consistent** (`h(s) ≤ c + h(s')`) ready, and be able to say why both of these are admissible on this maze: every move costs at least 1, so a distance in steps can never exceed a cost. Then look at `greedy_trap` — greedy expands only 60 nodes but returns a path costing 52 where the optimum is 25 — and be able to explain *why* ignoring `g(n)` does that. Report §6 also asks for A* with Manhattan vs Euclidean on `maze_big`: Euclidean is admissible but weaker (it underestimates more on this 4-connected grid), so it expands more nodes.

Done when. `greedy_trap -a gbfs` → 52, 25, 60; `maze_medium -a gbfs` → 80, 80, 81.

Q7 — A* Search · 3 marks · Nofil · ~45 min

What gets done. `aStarSearch(problem, heuristic)` — the shared loop with priority `f(n) = g(n) + h(n)`. With Manhattan it must return the cheapest path while expanding far fewer nodes than UCS.

How to prepare. Be able to state the guarantee precisely: A* in graph-search form returns an optimal path when the heuristic is **consistent**, and the reason Manhattan qualifies here is that every move costs at least 1. Compare `maze_big -a ucs -q` against `maze_big -a astar -H manhattan -q` and keep the numbers for the report.

Done when. `greedy_trap -a astar` → 25, 25, 155; `maze_big -a astar` → 124, 124, 679.

Q8 — KeyHuntProblem · 3 marks · Partner · ~3 h

What gets done. Fill in `getStartState`, `isGoalState` and `getSuccessors` in `griever_hole.py`. Thomas must step on **every** code piece (`K`), in any order, and then reach the exit `E`. The state representation is ours to choose, and that choice *is* the question. It must be **hashable** (tuples, frozensets, ints — never lists, sets or dicts), must be a **tuple whose first element is Thomas's `(row, col)`** because the display and grader read `state[0]`, and must be **small** — only what is needed to decide what happens next.

How to prepare. Answer the brief's own question before you code: *does the order in which pieces were collected matter for the future?* It does not — only **which** are still outstanding — so a `frozenset` of the uncollected keys is the right second element, giving `(position, frozenset_of_remaining_keys)`. A list of visited keys would be unhashable and would blow the state space up by a factor of 6! for nothing. In `getSuccessors`, use `self.maze.neighbors(position)` for the legal `(action, nextPosition)` pairs in N/S/E/W order and `self.maze.cost(nextPosition)` for the step cost, remove the entered cell from the remaining-keys set, and **keep the two bookkeeping lines** at the top of the method. The real test of the formulation is that the unmodified Q2 BFS solves it — if BFS breaks here, the state is wrong, not the search.

Done when. `keys_tiny -p keys -a bfs` → 26 steps, 490 expanded; `keys_small -p keys -a bfs` → 54 steps, 1723 expanded.

Q9 — Key-hunt heuristic · 4 marks · BOTH, paired · ~6–8 h over 3 days

What gets done. `keyHuntHeuristic(state, problem)` in `griever_hole.py`, graded by how few nodes A* expands on `keys_medium` against a UCS baseline of 9695. It must be **consistent** — `h(s) ≤ c + h(s')` for every state and successor, and `h = 0` at goal states — and it must be non-trivial (`h > 0` at the start). The grader checks consistency on the states A* visits; an inconsistent or trivial heuristic scores **0 for the whole question**, so correctness comes before strength at every step.

Nodes expanded on `keys_medium`	Marks
Consistent and non-trivial (`h > 0` at start)	1
≤ 7000	2
≤ 2600	3
≤ 1200	4

Our planned progression.

1. **Baseline (day 7).** When no keys remain, `h = mazeDistance(position, exit)`; otherwise take the **maximum over remaining keys** `k` of `mazeDistance(position, k) + mazeDistance(k, exit)`. This is admissible because any completed path from `position` must pass through every remaining `k` and finish at the exit, so its cost is at least that sum; and it is consistent because each term is a function of `position` alone that changes by at most 1 per step while any step costs at least 1. At a goal state no keys remain and `position == exit`, so `h = 0`. Expect this to clear the 2600 tier.

2. **Strengthen (days 8–9).** Add a spanning-tree style bound — `distance to the nearest remaining key + MST over the remaining keys + the smallest key-to-exit distance` — and take the **max** of it and the baseline. Re-run the consistency check after every change.

How to prepare. Know `mazeDistance(pos1, pos2, problem.maze, problem.heuristicInfo)`: it returns the **true number of steps** between two cells, ignoring terrain cost, and caches whole BFS layers into the dictionary you pass — so always pass `problem.heuristicInfo` or the heuristic will be unusably slow. Hold on to the brief's rule: **always take the `max` of admissible estimates, never their sum**, unless you can actually prove the sum is admissible — that proof is the whole trick in the MST bound, so if we cannot argue it, we do not ship it. Both of us must be able to state the consistency argument out loud, since this is the most likely viva question in the project.

6. The report · 5 marks

PDF, short and well formatted, no code screenshots. Use `--compare` to collect the numbers.

§	Content	Owner
1	**AI assistant used** — name the tool and briefly describe what we used it for	Nofil
2	**Results tables** — `python runner.py -l <maze> --compare` on `maze_medium`, `open_glade`, `ivy_maze`	Partner runs, Nofil typesets
3	**DFS vs BFS** — why the DFS path on `maze_medium` is so long; which is complete, which is optimal, and under what conditions	Partner drafts
4	**IDS** — why it is fast on `maze_small` and hopeless on `open_glade` though both are small, in terms of maze structure and our path-based cycle check; and what IDS buys over BFS in return	Partner drafts
5	**Costs** — why BFS and UCS disagree on `griever_alley`; on which mazes they are guaranteed to tie	Partner drafts
6	**Heuristics** — what goes wrong for greedy on `greedy_trap`; is Euclidean admissible here; A* with Manhattan vs Euclidean on `maze_big`, with the node counts and the reason for the gap	Nofil
7	**Our Q9 heuristic** — describe it, argue consistency, report nodes expanded	Nofil

7. Submission checklist — Monday 13 October

1. `git pull` on a **fresh clone** of `main` in a clean directory.
2. `./verify.sh` — every row of the §2 table matches.
3. `python runner.py -l keys_medium -p keys -a astar -H keyhunt` — confirm the expansion count and which mark tier it lands in.
4. Three `--compare` runs re-done with the **exact files being submitted**; tables in the report match.
5. Report: no code screenshots anywhere; AI-assistant section present; all 7 sections answered.
6. Zip contains exactly `search.py`, `heuristics.py`, `griever_hole.py`, `report.pdf` — no `__pycache__`, no `.DS_Store`, no helper scripts.
7. Named `<group_number>_project1.zip`, with the group number from GCR.
8. One of us uploads; the other confirms the upload is visible on GCR.
9. Both of us have re-read the other's code once more for the viva.

8. Risks

Risk	Mitigation
Q9 stalls below the 1200 tier	Tiered marks mean partial credit is banked early; secure consistency + `h > 0` on day 7, then improve. Three days are allocated, and 6 Oct can slip into it.
DLS returns the wrong one of `CUTOFF` / `None`	The two `maze_small` rows (`-d 60` and `-d 30`) catch exactly this; also hand-test the brief's `A→B→C→D→G` graph with `GraphProblem`.
Merge conflicts in `search.py`	Function-level ownership, one branch per question, partner rebases on `main` before each branch.
Numbers nearly but not quite matching the table	Re-check the five conventions in §2 — goal-test on pop, successor order, mark-expanded-on-pop, one `getSuccessors` per expansion, graph not tree.
Viva exposes one partner not knowing the other's half	12 Oct is reserved for cross-explaining; each of us presents the code we did **not** write.
Similarity check flags our work	Write everything ourselves, disclose AI use honestly in report §1, never paste an existing Pacman-style solution.